

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 3

Student Name: M.Purna Chandra

Roll No.: A24126552095

Date: 11-08-2026

1. TITLE

Arrays and Functions in JavaScript

2. AIM

To design and implement a JavaScript program that uses arrays and functions to perform operations such as adding numbers, displaying all elements, computing the sum and average, removing a specific element, and clearing the entire array.

3. OBJECTIVE

The objectives of this experiment are:

- To understand how to declare and initialise arrays in JavaScript.
 - To define reusable functions that operate on a shared global array.
 - To use array methods `push()`, `indexOf()`, and `splice()` for adding, searching, and removing elements.
 - To implement a for loop to iterate over array elements and compute their sum.
 - To compute and format the average of array elements using arithmetic and `toFixed()`.
 - To handle edge cases such as an empty array in the average function to prevent division by zero.
 - To use `console.log()` with template literals to display results of each operation.
-

4. THEORY

JavaScript Arrays — ordered, zero-indexed collections that can hold any number of values. Arrays are declared with `[]` and grow dynamically. Elements are accessed by index and can be added or removed using built-in methods.

Array Methods — `push(value)` appends an element to the end of the array; `indexOf(value)` returns the index of the first match or -1 if absent; `splice(index, count)` removes count elements starting at index, modifying the array in-place.

Functions — reusable blocks of code declared with the function keyword. Functions can accept parameters, perform operations on global or local variables, log output to the console, and return values for use by other functions.

for Loop — a control structure that repeats a block of code a fixed number of times. Iterating with `for (let i = 0; i < array.length; i++)` visits every element of an array by index.

Edge Case Handling — checking a condition before performing a potentially unsafe operation. In `getAverage()`, `numbers.length === 0` is tested before division to return 0 immediately and avoid a division-by-zero result.

5. ALGORITHM / PROCEDURE

- Create a JavaScript file named `arrayfunctions.js`.
 - Declare a global array `let numbers = []` to store numeric values shared by all functions.
 - Define `addNumber(num)`: call `numbers.push(num)` to append the value and log "Added: num" to the console.
 - Define `displayAll()`: log the array joined with " | " as a separator using `numbers.join(" | ")`.
 - Define `getSum()`: initialise `sum = 0`, loop through the array with a for loop adding each element, log and return the total.
 - Define `getAverage()`: return 0 if the array is empty; otherwise call `getSum()`, divide by `numbers.length`, format to two decimal places with `toFixed(2)`, log and return the result.
 - Define `removeNumber(num)`: use `indexOf()` to find the element; if found call `splice()` to remove it and log "Removed: num"; otherwise log "num not found".
 - Define `clearAll()`: reset `numbers = []` and log "Cleared all numbers".
 - Call `addNumber()` five times with values 8, 3, 15, 6, 22 to populate the array.
 - Call `displayAll()`, `getSum()`, and `getAverage()` to verify the stored values and calculations.
 - Call `removeNumber(15)` then `displayAll()` to verify the element was removed.
 - Call `clearAll()` then `displayAll()` to verify the array is empty.
 - Run the file in a browser console or Node.js and record expected vs. actual output for each test case.
-

6. PROGRAM

arrayfunctions.js

// Arrays and Functions

```
let numbers = [];
```

```
function addNumber(num) {
```

```
    numbers.push(num);
    console.log(`Added: ${num}`);
}

function displayAll() {
    console.log("\nNumbers:", numbers.join(" | "));
}

function getSum() {
    let sum = 0;
    for (let i = 0; i < numbers.length; i++) {
        sum += numbers[i];
    }
    console.log(`Sum = ${sum}`);
    return sum;
}

function getAverage() {
    if (numbers.length === 0) {
        console.log("Average = 0\n");
        return 0;
    }
    let sum = getSum();
    let avg = sum / numbers.length;
    console.log(`Average = ${avg.toFixed(2)}\n`);
    return avg;
}

function removeNumber(num) {
    let idx = numbers.indexOf(num);
    if (idx !== -1) {
        numbers.splice(idx, 1);
        console.log(`Removed: ${num}`);
    } else {
        console.log(`${num} not found`);
    }
}
```

```
    }  
}  
  
function clearAll() {  
    numbers = [];  
    console.log("Cleared all numbers");  
}  
  
addNumber(8);  
addNumber(3);  
addNumber(15);  
addNumber(6);  
addNumber(22);  
  
displayAll();  
  
getSum();  
  
getAverage();  
  
removeNumber(15);  
  
displayAll();  
  
removeNumber(99);  
displayAll()  
  
clearAll();  
displayAll();  
  
getAverage();
```

7. CODE EXPLANATION

Code	Explanation
let numbers = []	Declares a global array named numbers, initially empty, that all functions operate on.
numbers.push(num)	Appends the value of num to the end of the numbers array inside addNumber().
numbers.join(" ")	Joins all array elements into a single string separated by " " for display in displayAll().
for loop in getSum()	Iterates through every element of the array using an index, accumulating a running total in sum.
sum += numbers[i]	Adds the element at index i to the running sum on each iteration of the loop.
numbers.length === 0	Guard check in getAverage() that returns 0 immediately if the array is empty, preventing division by zero.
avg.toFixed(2)	Formats the computed average to exactly two decimal places before logging it.
numbers.indexOf(num)	Returns the index of the first occurrence of num in the array, or -1 if it is not found.
numbers.splice(idx, 1)	Removes one element at position idx from the array in-place inside removeNumber().
numbers = []	Resets the global numbers array to an empty array inside clearAll(), discarding all stored values.
console.log(...)	Prints messages and results to the console for each operation throughout the program.
return sum / return avg	Each calculation function returns its computed value so it can be reused by other functions, e.g. getAverage() calls getSum().

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Use addNumber(num)	Console prints Added: number ;	All five values logged correctly	Pass
TC-02	Call displayAll() after adding numbers	Console prints Numbers	Array displayed with correct separator	Pass
TC-03	Call getSum() with numbers	Console prints Sum = 54	Sum computed and logged as 54	Pass
TC-04	Call getAverage() with numbers	Console prints Sum = 54 then Average = 10.80	Average computed and logged as 10.80	Pass

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-05	Call removeNumber(number) when 15 is in the array	Console prints Removed: 15 and array becomes [8, 3, 6, 22]	15 removed successfully	Pass
TC-06	Call displayAll() after removing 15	Console prints Numbers: 8 3 6 22	Array displayed without 15	Pass
TC-07	Call removeNumber(99) when 99 is not in the array	Console prints 99 not found	Not-found message logged correctly	Pass
TC-08	Call clearAll()	Console prints Cleared all numbers and array becomes []	Array cleared successfully	Pass
TC-09	Call displayAll() after clearAll()	Console prints Numbers: (empty string)	Empty array displayed with no elements	Pass
TC-10	Call getAverage() on empty array	Function returns 0 without calling getSum()	Returned 0 with no division error	Pass

9. OUTPUT

Output 1: Adding Numbers

Five calls to addNumber() separate lines as each value is pushed into the array.

```
PS C:\Users\Purna\Documents\Projects\FullStack\Records\week_3> node arrayfunctions.js
Added: 8
Added: 3
Added: 15
Added: 6
Added: 22
```

Output 2: Display, Sum, and Average

displayAll() prints "Numbers: 8 | 3 | 15 | 6 | 22". getSum() prints "Sum = 54". getAverage() first prints "Sum = 54" (by calling getSum() internally) then prints "Average = 10.80".

```
Numbers: 8 | 3 | 15 | 6 | 22
Sum = 54
Sum = 54
Average = 10.80
```

Output 3: Remove and Display

removeNumber(15) prints "Removed: 15". The subsequent displayAll() prints "Numbers: 8 | 3 | 6 | 22", confirming 15 has been removed from the array.

```
Removed: 15  
Numbers: 8 | 3 | 6 | 22
```

Output 4: Remove and Display

removeNumber(99) prints "99 not found". The subsequent displayAll() prints "Numbers: 8 | 3 | 6 | 22".

```
99 not found  
  
Numbers: 8 | 3 | 6 | 22
```

Output 5: Clear and Display

clearAll() prints "Cleared all numbers". The final displayAll() prints "Numbers: " with an empty string, confirming the array has been reset to [].

```
Cleared all numbers  
  
Numbers:
```

Output 5: Average on empty Array

```
Average = 0
```

10. RESULT

Thus, a JavaScript program using arrays and functions was successfully designed and implemented. The program demonstrates array declaration, element addition and removal using push(), indexOf(), and splice(), sum and average computation using a for loop, edge case handling for an empty array, and output formatting using template literals and console.log(). All ten test cases passed successfully, confirming correct behaviour of all functions.